TEMA 3: ARQUITECTURA DEL CONJUNTO DE INSTRUCCIONES

3.1.- Conceptos de arquitectura de computación.

3.2.- Direccionamiento de operandos.

3.3.- Modos de direccionamiento.

3.4.- Arquitecturas del conjunto de instrucciones.

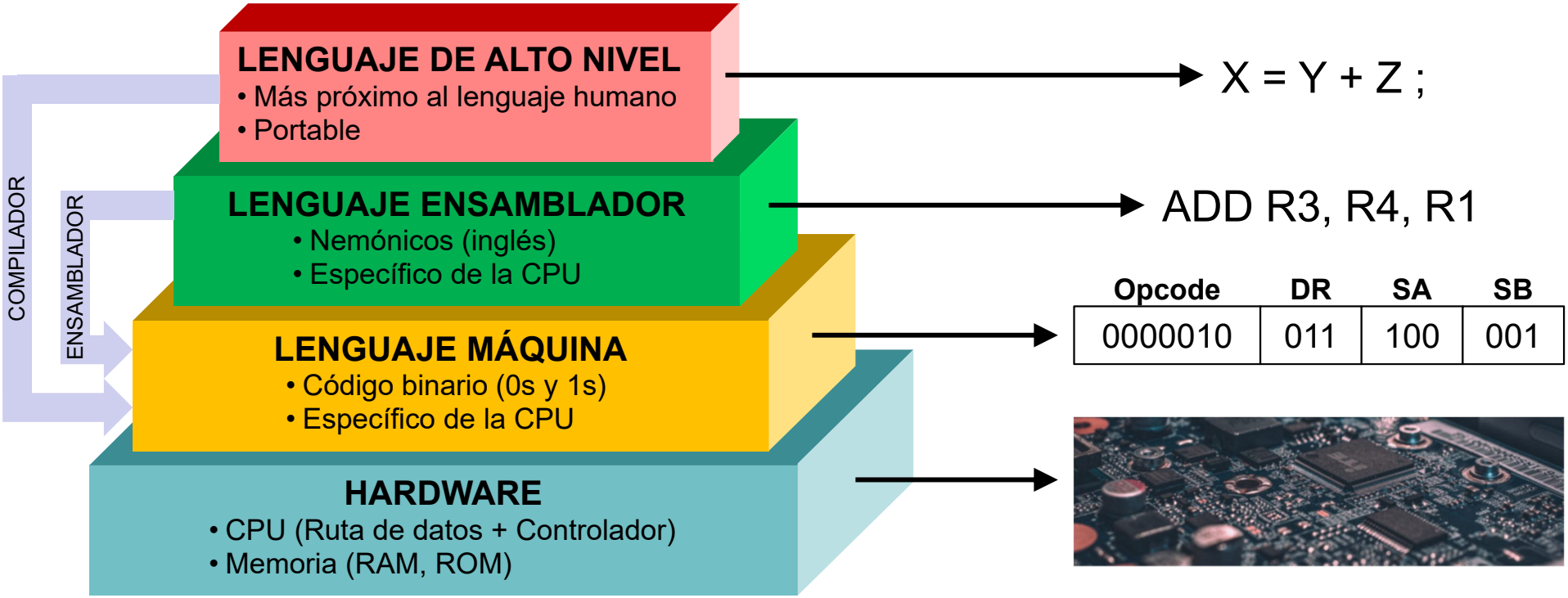
3.5.- Instrucciones de transferencia de datos.

3.6.- Instrucciones de manipulación de datos.

3.7.- Instrucciones de control de programa.

3.8.- Interrupciones.

ARQUITECTURA DE COMPUTACIÓN



La colección de todas las instrucciones que puede ejecutar un procesador se denomina *conjunto de instrucciones*, y a su descripción, *arquitectura del conjunto de instrucciones*.

La arquitectura del conjunto de instrucciones determina cómo debe ser el hardware del procesador y cómo debe estar organizado.

Los manuales de usuario de los procesadores se centran en la descripción del conjunto de instrucciones que pueden ejecutar, así como de los elementos del procesador que son accesibles al programador.

CICLO DE OPERACIÓN BÁSICO DE UN PROCESADOR

La unidad de control de un procesador se diseña para ejecutar cada una de las instrucciones de un programa efectuando la siguiente secuencia de pasos:

1.- Obtener la instrucción de memoria. Almacenarla en un registro de control.
2.- Decodificar la instrucción.
3.- Localizar los operandos empleados en la instrucción.
4.- Obtener de la memoria los operandos (si fuese necesario)
5.- Ejecutar la operación en la ruta de datos.
6.- Almacenar el resultado en un lugar adecuado.
7.- Volver al paso 1 para procesar la siguiente instrucción.

La unidad de control del procesador consta de un registro especial, el *contador de programa* PC (Program Counter). Su contenido apunta a la posición de memoria de la instrucción que se va a ejecutar a continuación, y se incrementa cada vez que se lee una instrucción del programa almacenado en la memoria.

La decodificación del paso 2 determina la operación a ejecutar y los modos de direccionamiento de la instrucción.

En el paso 3, los operandos se localizan según los modos de direccionamiento y los campos de direcciones de la instrucción.

El procesador ejecuta la instrucción sobre los operandos, almacena el resultado y regresa al paso 1 para obtener la siguiente instrucción del programa.

CONJUNTO DE REGISTROS

El conjunto de registros está formado por todos los registros del procesador accesibles al programador:

- El contador de programa PC.
- Banco de registros de la ruta de datos.
- El *puntero de pila* SP (Stack Pointer). Permite definir estructuras LIFO en la memoria.
- El *registro de estado del procesador* PSR (Processor Status Register). Almacena el estado o modo de funcionamiento en que se encuentra el procesador (*usuario*, *supervisor*, etc.), e incluye los bits C, N, V y Z de la ALU. Estos bits, normalmente denominados *banderines* (flags), pueden utilizarse para tomar decisiones que determinen el flujo de ejecución del programa.

Los procesadores constan además de una serie de registros que normalmente no son accesibles al programador:

- El registro de instrucción IR.
- El registro CAR.
- Registros ocultos del banco de registros de la ruta de datos. Sólo son accesibles a los microprogramas para, por ejemplo, almacenar resultados parciales durante la ejecución de una instrucción.
- Los registros de pipeline en procesadores segmentados.

TEMA 3: ARQUITECTURA DEL CONJUNTO DE INSTRUCCIONES

3.1.- Conceptos de arquitectura de computación.

3.2.- Direccionamiento de operandos.

3.3.- Modos de direccionamiento.

3.4.- Arquitecturas del conjunto de instrucciones.

3.5.- Instrucciones de transferencia de datos.

3.6.- Instrucciones de manipulación de datos.

3.7.- Instrucciones de control de programa.

3.8.- Interrupciones.

En una instrucción de manipulación de datos, como por ejemplo la instrucción ADD que vimos en el tema anterior, se necesita especificar tanto los datos que se van a sumar, como el destino del resultado. Esto se conoce como *direccionamiento de operandos*. Dichos operandos pueden encontrarse tanto en el banco de registros como en la memoria.

Si los operandos se encuentran en la memoria, los campos de dirección de la instrucción necesitan muchos más bits que si los operandos se encuentran en registros.

Ejemplo: Para una memoria de 1 Giga, se necesitan 30 bits de dirección.
Para un procesador con 32 registros, se necesitan 5 bits de dirección.

El número de operandos que se pueden direccionar en una instrucción es un factor importante a tener en cuenta a la hora de definir la arquitectura del conjunto de instrucciones de un procesador:

- Instrucciones de 3 direcciones: Ej. ADD R1, R2, R3 (operación R1 ← R2 + R3)

- Instrucciones de 2 direcciones: Ej. ST R1, R0 (operación M[R1] ← R0)

- Instrucciones de 1 dirección: Ej. JMP AD (operación PC ← PC + se AD)

En general, cuantos más operandos se direccionen en una instrucción, mayor espacio en memoria ocupará la instrucción.

TEMA 3: ARQUITECTURA DEL CONJUNTO DE INSTRUCCIONES

3.1.- Conceptos de arquitectura de computación.

3.2.- Direccionamiento de operandos.

3.3.- Modos de direccionamiento.

3.4.- Arquitecturas del conjunto de instrucciones.

3.5.- Instrucciones de transferencia de datos.

3.6.- Instrucciones de manipulación de datos.

3.7.- Instrucciones de control de programa.

3.8.- Interrupciones.

DEFINICIÓN

El *modo de direccionamiento* especifica una regla para interpretar o modificar el campo de direcciones de la instrucción antes de que se haga realmente referencia al operando. A la dirección del operando generada mediante la aplicación de esa regla se la denomina *dirección efectiva*.

Formato de la instrucción:

Opcode	Modo	Direcciones u operandos

Los procesadores utilizan técnicas de modo de direccionamiento para ajustarse a las siguientes características:

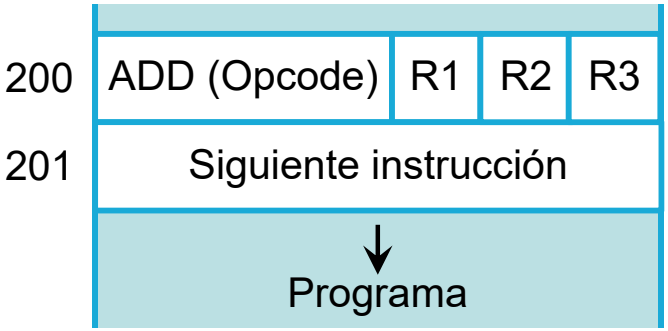
- Proporcionar flexibilidad al usuario en la programación mediante punteros a memoria, contadores para el control de bucles, indexar datos y reubicar programas.
- Reducir el número de bits de los campos de direcciones de la instrucción.

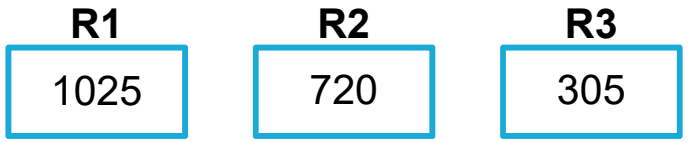
Disponer de varios modos de direccionamiento proporciona al programador experimentado la posibilidad de escribir programas que requieran pocas instrucciones. Sin embargo, el uso de modos de direccionamiento complejos puede implicar un mayor tiempo de ejecución.

MODO REGISTRO

El campo de dirección de la instrucción especifica un registro del procesador en lugar de una posición de memoria. El operando está contenido en el registro direccionado en la instrucción.

Ejemplo: Suma ADD R1, R2, R3



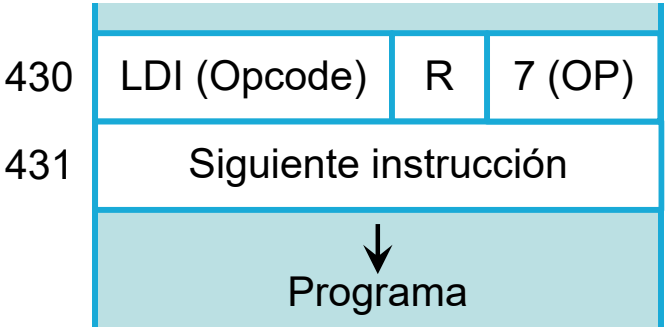


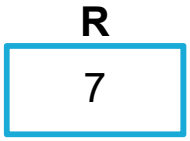
Operación: R1 ← R2+R3 = 1025
PC ← 201

MODO INMEDIATO

El operando se especifica directamente en la instrucción. En vez de tener un campo de direcciones, la instrucción posee un campo de operando inmediato.

Ejemplo: Inicialización de un registro con un operando inmediato LDI R, #OP





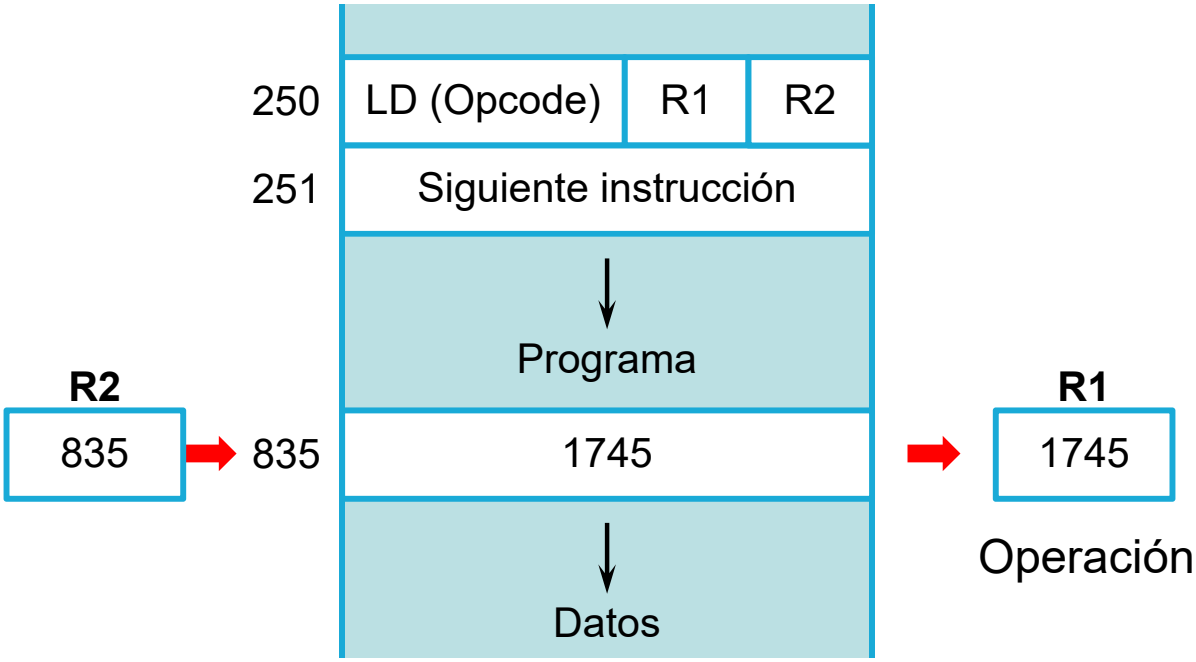
Operación: R ← OP = 7
PC ← 431

MODO REGISTRO INDIRECTO

La instrucción direcciona un registro del procesador en el que está la dirección de memoria donde se encuentra el operando.

Para acceder al operando correctamente, el programador debe asegurarse de que la dirección efectiva esté ya disponible en el registro.

Ejemplo: Transferencia de memoria a registro LD R1, [R2]



Operación: R1 ← M[R2] = 1745
PC ← 251

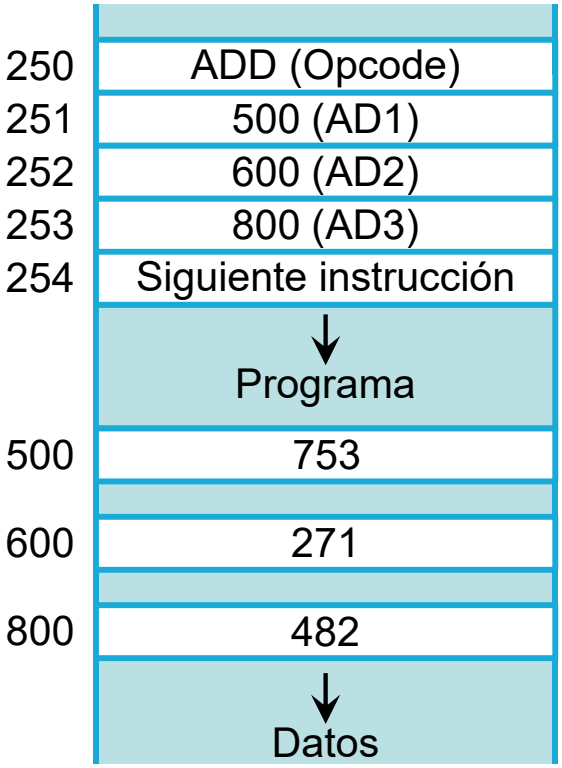
La ventaja de este modo de direccionamiento es que el campo de direcciones de la instrucción necesita menos bits que los que serían necesarios para especificar una dirección de memoria directamente.

MODO DE DIRECCIONAMIENTO DIRECTO

El campo de dirección de la instrucción proporciona directamente la dirección efectiva.

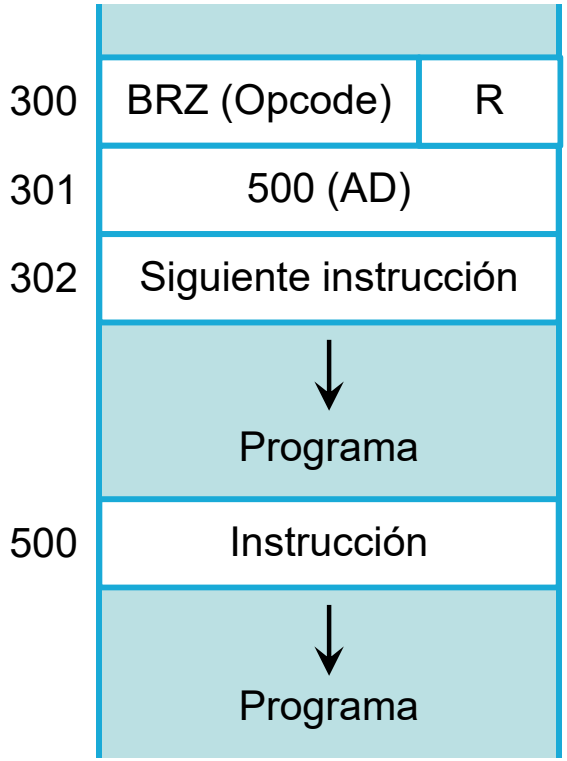
Ejemplos:

(a) Suma ADD [AD1], [AD2], [AD3]



Operación: M[500] ← 271+482 = 753
PC ← 254

(b) Salto condicional BRZ R, [AD]

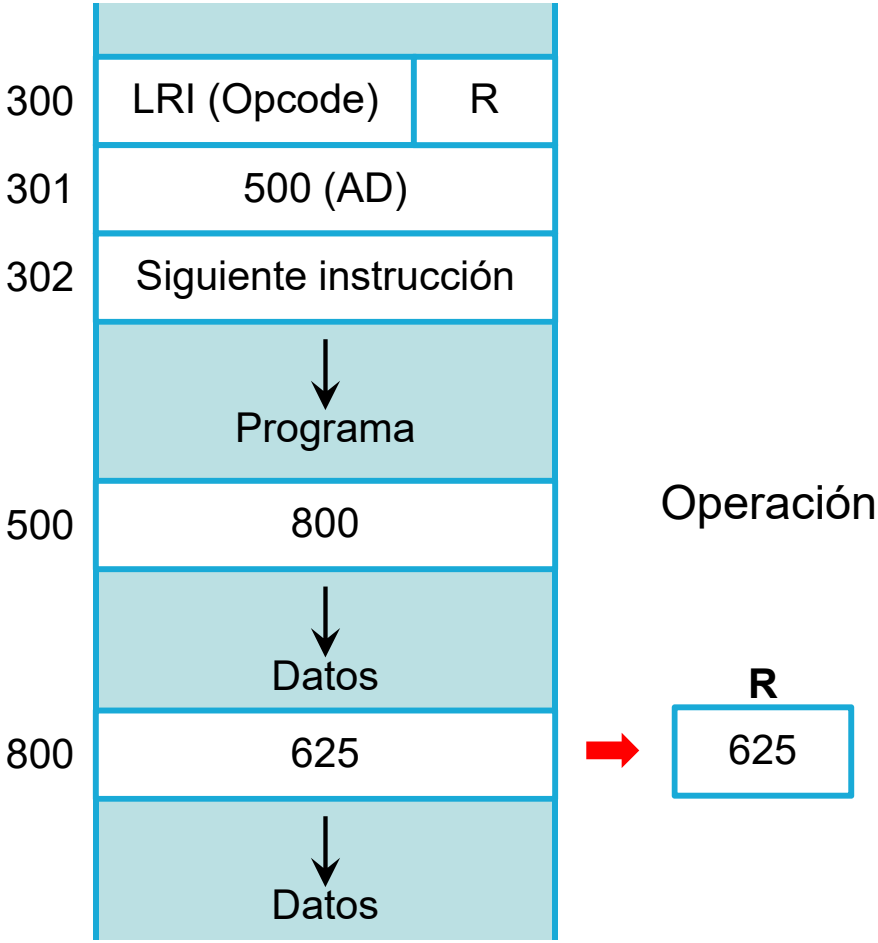


Operación: PC ← 500 si R=0
PC ← 302 si R≠0

MODO DE DIRECCIONAMIENTO INDIRECTO

El campo de direcciones de la instrucción proporciona la dirección en que se encuentra la dirección efectiva.

Ejemplo: Transferencia de datos de memoria a registro indirecta LRI R, [AD]



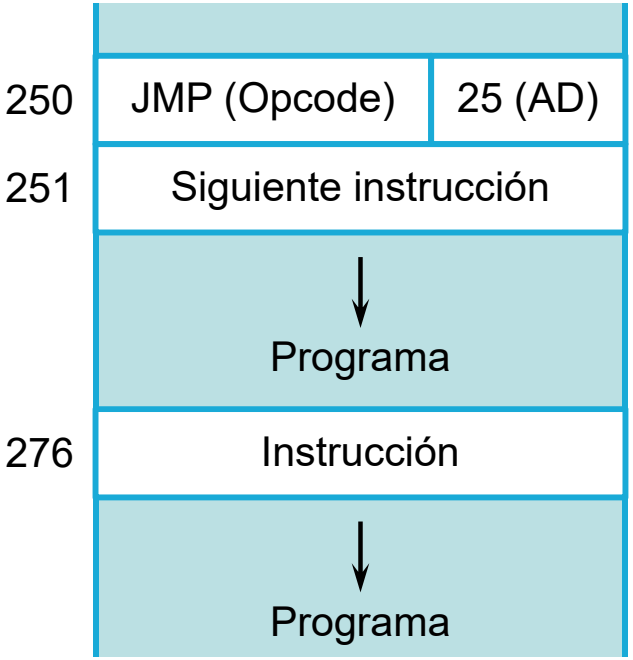
Operación: $R \leftarrow M[M[500]] = M[800] = 625$

$PC \leftarrow 302$

MODO DE DIRECCIONAMIENTO RELATIVO

La dirección efectiva se obtiene como suma de la parte de dirección de la instrucción y el contenido del PC. La parte de dirección de la instrucción es un número con signo.

Ejemplo: Salto incondicional JMP AD



Operación: PC ← 251+25 = 276

Este tipo de direccionamiento se usa frecuentemente en instrucciones de salto cuando la dirección de bifurcación está cercana a la instrucción de salto. Este direccionamiento genera instrucciones más compactas, ya que la dirección relativa necesita de menos bits que una dirección de memoria.

MODO INDEXADO O DE REGISTRO BASE

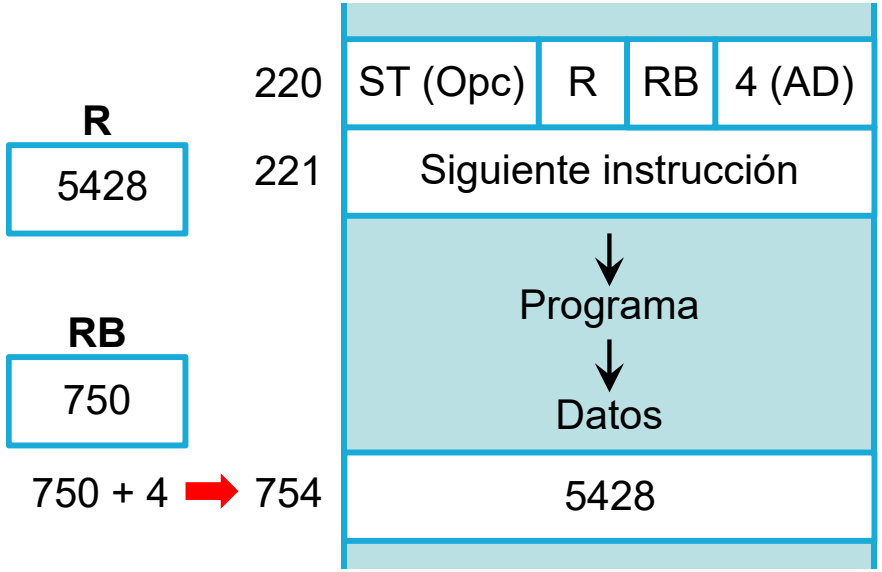
La dirección efectiva se obtiene como suma de la parte de dirección de la instrucción (desplazamiento) y el contenido de un registro del procesador, llamado *registro base*.

Se usa cuando se desea acceder o manipular un array de datos almacenado en la memoria:

- El contenido del registro base marca el comienzo del array.
- El desplazamiento marca la distancia entre el comienzo y la dirección del operando accedido.

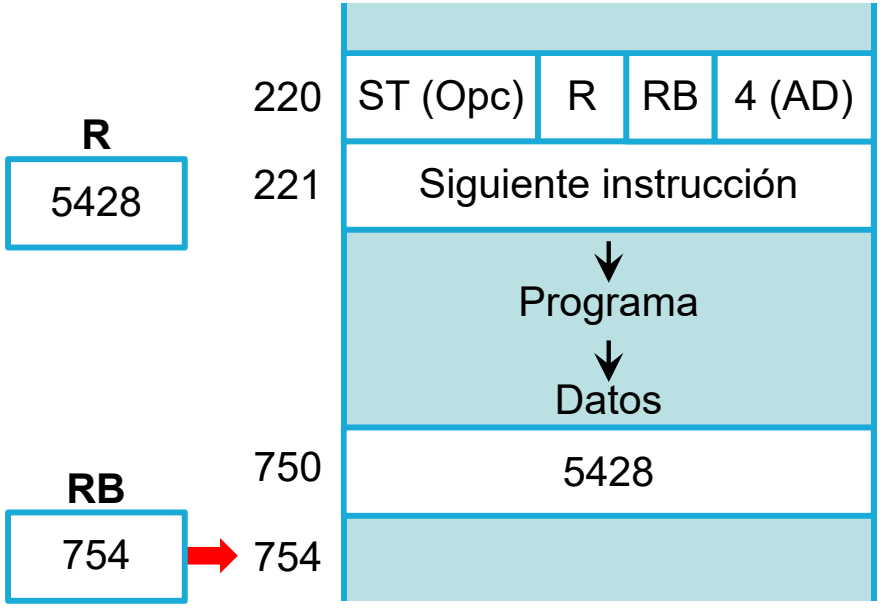
Ejemplo: Almacenar el contenido de un registro en la memoria (ST):

a) Pre-indexado ST R, [RB, #AD]



Operación: M[754] ← 5428
PC ← 221

b) Post-indexado ST R, [RB], #AD



Operación: M[750] ← 5428
PC ← 221
RB ← 750 + 4 = 754

TEMA 3: ARQUITECTURA DEL CONJUNTO DE INSTRUCCIONES

3.1.- Conceptos de arquitectura de computación.

3.2.- Direccionamiento de operandos.

3.3.- Modos de direccionamiento.

3.4.- Arquitecturas del conjunto de instrucciones.

3.5.- Instrucciones de transferencia de datos.

3.6.- Instrucciones de manipulación de datos.

3.7.- Instrucciones de control de programa.

3.8.- Interrupciones.

	ARQUITECTURA RISC *Reduced Instruction Set Computer*	**ARQUITECTURA CISC** *Complex Instruction Set Computer*
Instrucciones	Sólo realizan operaciones elementales	Realizan tanto operaciones elementales como complejas
Accesos a memoria	Sólo en las instrucciones de carga y almacenamiento	Disponibles en prácticamente todos los tipos de instrucciones
Modos de direccionamiento	Limitados	Muchos disponibles
Formato de las instrucciones	Todas de la misma longitud	Diferentes longitudes

Características de la arquitectura RISC:

- Su objetivo es conseguir un alto rendimiento (alta velocidad de ejecución).
- Necesita un banco de registros relativamente grande.
- Al tener instrucciones de longitud fija, modos de direccionamiento limitados, y realizar sólo operaciones básicas, la unidad de control es relativamente simple, y típicamente está cableada.

Características de la arquitectura CISC:

- Su objetivo es proporcionar instrucciones que faciliten realizar programas compactos
- Debido a la alta accesibilidad de la memoria, el banco de registros es reducido.
- Debido a la complejidad de las instrucciones y la diversidad de sus formatos, la unidad de control es compleja y suele utilizar microprogramación.

ARQUITECTURAS DE DIRECCIONAMIENTO

Memoria a memoria

Todas las instrucciones pueden acceder directamente a la memoria.

Registro a registro o de carga/almacenamiento

Las instrucciones de manipulación de datos operan sobre registros. Sólo se accede a la memoria en las de carga y almacenamiento.

Ejemplo: Programa para calcular X = (A+B)×(C+D)

ADD [T1], [A], [B]	M[T1] ← M[A]+M[B]
ADD [T2], [C], [D]	M[T2] ← M[C]+M[D]
MUL [X], [T1], [T2]	M[X] ← M[T1]×M[T2]

- Los programas son sencillos.
- El elevado número de accesos a memoria ralentiza su ejecución.
- El que las instrucciones accedan directamente a memoria hace que las estructuras de control de la CPU sean complejas (arquitectura CISC).

LD	R1, [A]	R1 ← M[A]
LD	R2, [B]	R2 ← M[B]
ADD	R3, R1, R2	R3 ← R1 + R2
LD	R1, [C]	R1 ← M[C]
LD	R2, [D]	R2 ← M[D]
ADD	R1, R1, R2	R1 ← R1 + R2
MUL	R1, R1, R3	R1 ← R1 × R3
ST	[X], R1	M[X] ← R1

- Los programas son más largos.
- Al reducirse los accesos a memoria su ejecución es más rápida.
- Las estructuras de control de la CPU son más sencillas (arquitectura RISC).

TEMA 3: ARQUITECTURA DEL CONJUNTO DE INSTRUCCIONES

3.1.- Conceptos de arquitectura de computación.

3.2.- Direccionamiento de operandos.

3.3.- Modos de direccionamiento.

3.4.- Arquitecturas del conjunto de instrucciones.

3.5.- Instrucciones de transferencia de datos.

3.6.- Instrucciones de manipulación de datos.

3.7.- Instrucciones de control de programa.

3.8.- Interrupciones.

Las instrucciones de transferencia de datos mueven un dato de un lugar del procesador a otro, sin modificarlo.

Las típicas transferencias son entre memoria y los registros del procesador, entre estos y los registros de entrada/salida (E/S), y entre los propios registros de procesador:

Nombre	Nemónico	Descripción
Load	LD	Desde una posición de memoria a un registro del procesador.
Store	ST	Desde un registro del procesador a una posición de memoria.
Move	MOV	Desde un registro del procesador a otro. También se utiliza para transferir datos entre dos posiciones de memoria.
Exchange	XCH	Intercambia datos entre dos registros del procesador o entre dos posiciones de memoria.
Push	PUSH	Transferencia de datos con la pila
Pop	POP	
Input	IN	Transferencia de datos con registros E/S
Output	OUT	

TRANSFERENCIA DE DATOS CON LA PILA

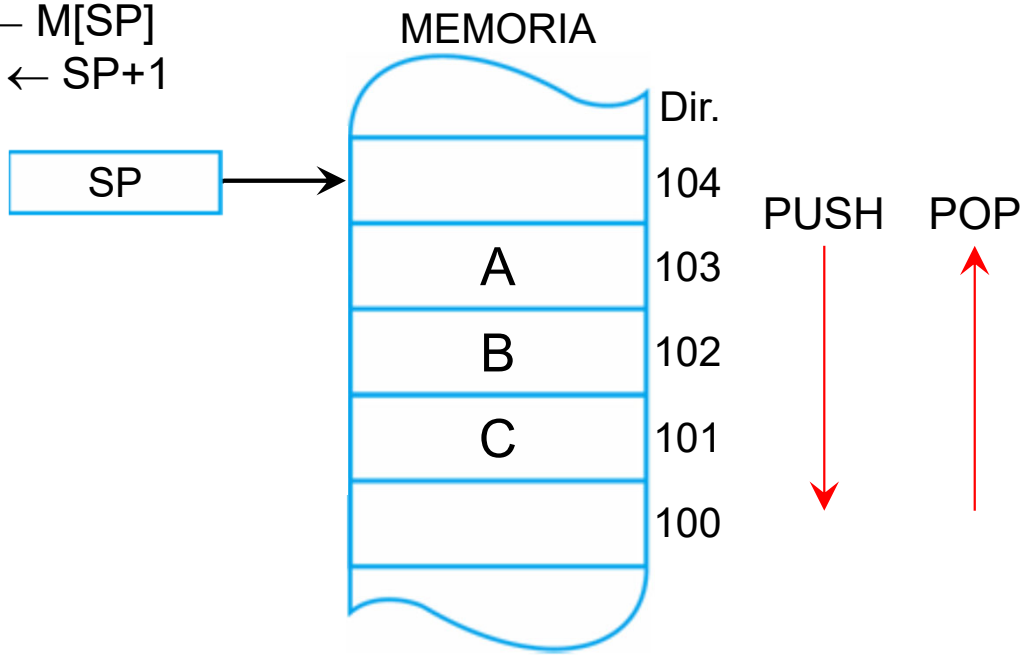
La arquitectura basada en pila posee características que facilitan ciertos tipos de procesamiento de datos y de control de tareas. Así, en algunas calculadoras y procesadores se usan pilas para la evaluación de expresiones aritméticas.

Las pilas en memoria se organizan mediante un registro llamado *puntero de pila* SP (Stack Pointer). La dirección apuntada por SP recibe el nombre de *tope de pila* TOS (Top Of Stack).

PUSH es la instrucción que almacena datos en la pila, y POP la que los extrae de ella:

PUSH R: SP ← SP-1
M[SP] ← R

POP R: R ← M[SP]
SP ← SP+1



<u>Programa de ejemplo:</u>

PUSH R1 (R1 contiene A)

PUSH R2 (R2 contiene B)

PUSH R3 (R3 contiene C)

POP R1 (C se graba en R1)

POP R2 (B se graba en R2)

POP R3 (A se graba en R3)

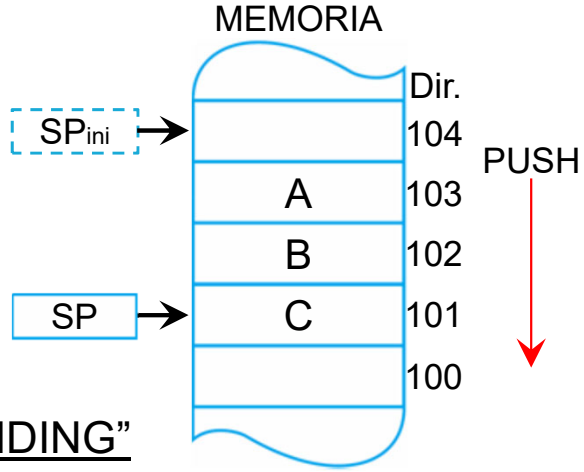
SP se inicializa con la dirección de memoria donde quiera iniciarse la pila (104 en el ejemplo).

TIPOS DE PILAS

Esquemas de las pilas en memoria después de haber ejecutado las tres instrucciones PUSH:

PUSH R:
SP ← SP-1
M[SP] ← R

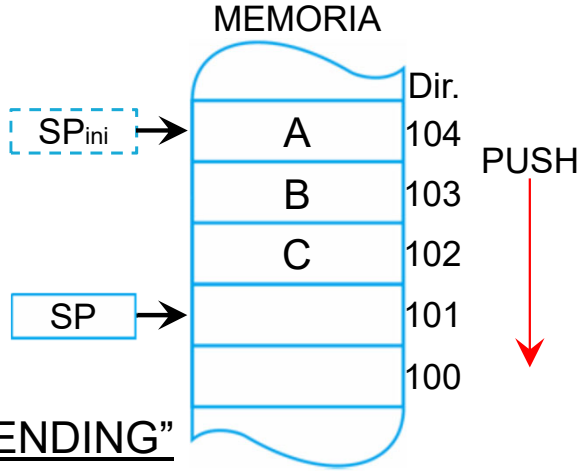
POP R:
R ← M[SP]
SP ← SP+1



"FULL DESCENDING"

PUSH R:
M[SP] ← R
SP ← SP-1

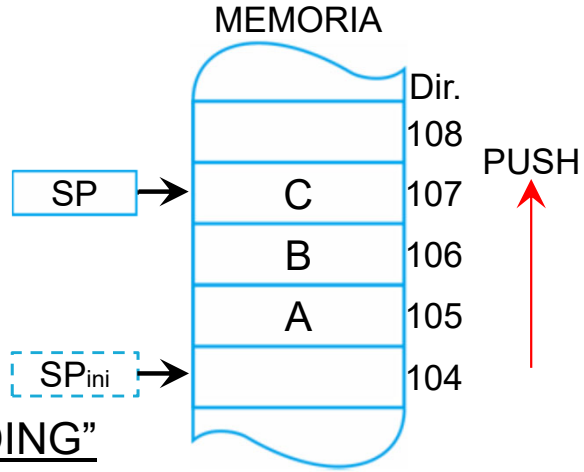
POP R:
SP ← SP+1
R ← M[SP]



"EMPTY DESCENDING"

PUSH R:
SP ← SP+1
M[SP] ← R

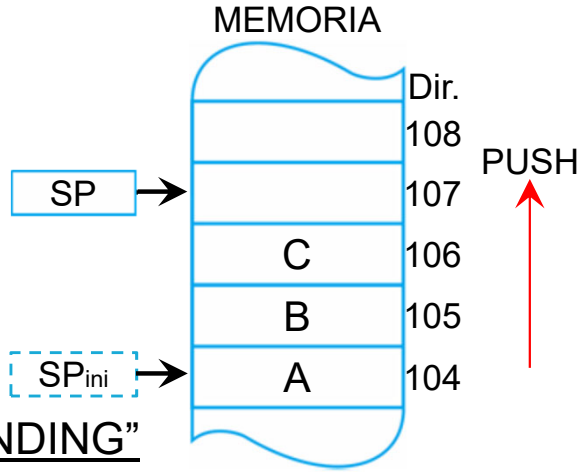
POP R:
R ← M[SP]
SP ← SP-1



"FULL ASCENDING"

PUSH R:
M[SP] ← R
SP ← SP+1

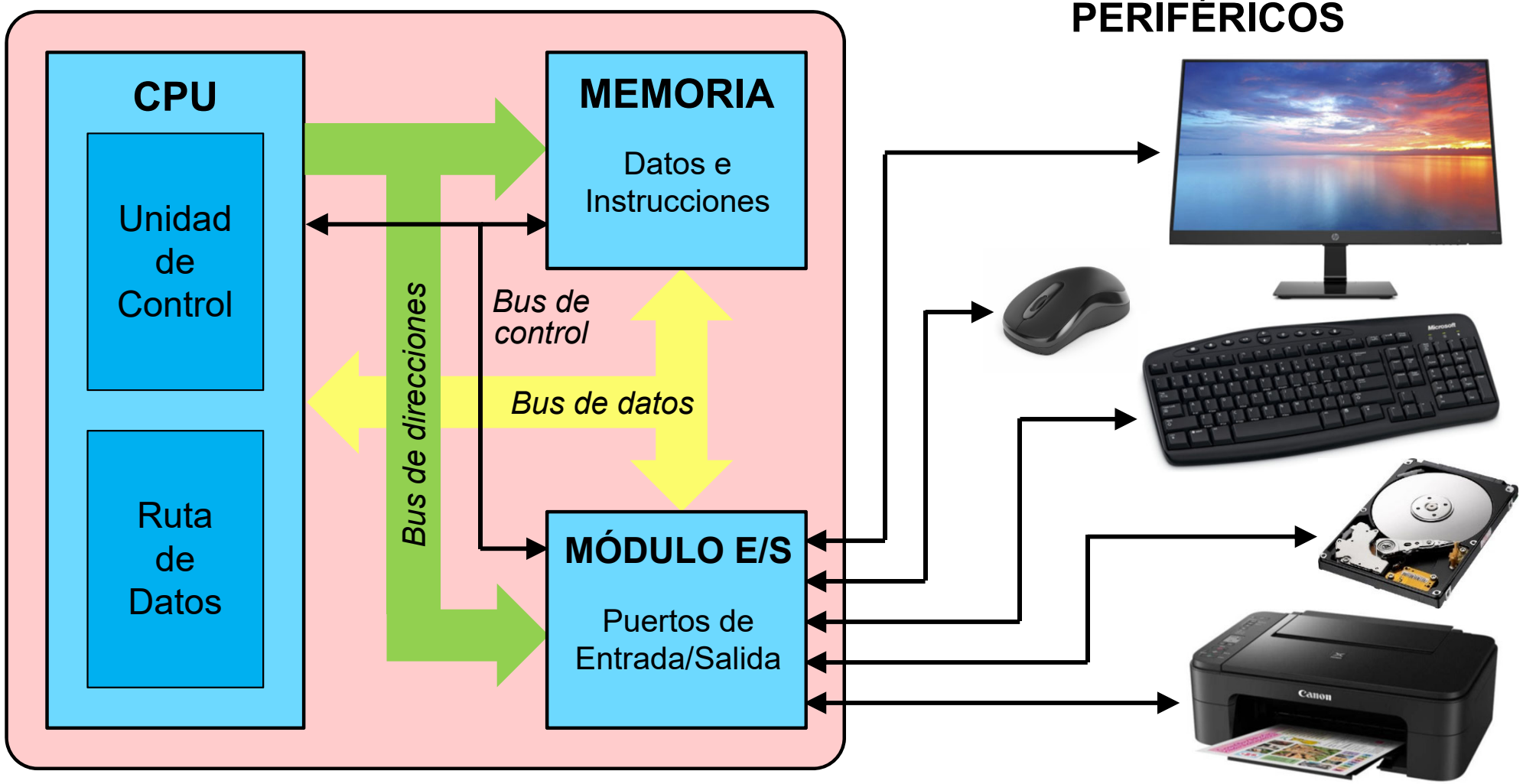
POP R:
SP ← SP-1
R ← M[SP]



"EMPTY ASCENDING"

TRANSFERENCIA DE DATOS CON REGISTROS E/S

Los registros E/S, llamados comúnmente *puertos*, son los que utiliza un procesador para intercambiar datos con dispositivos externos (*periféricos*).



TRANSFERENCIA DE DATOS CON REGISTROS E/S

Normalmente cada puerto disponible tiene asociado un código de dirección. Las direcciones de los puertos pueden asignarse de dos formas:

a) Sistemas con E/S independiente o separada.

- El rango de direcciones asignadas a la memoria y a los puertos de E/S son independientes entre sí.
- El procesador tiene instrucciones de transferencia de datos diferentes, según se trate de transferencias con la memoria o con los puertos E/S: LD y ST para la memoria, e IN y OUT para los puertos.

b) Sistemas con E/S mapeada o ubicada en memoria.

- Se asigna un subrango de direcciones de memoria para direccionar los puertos E/S: cada puerto E/S se trata como una posición de memoria más.
- El procesador no tiene instrucciones de transferencia de datos específicas para E/S, se usan las mismas que las de la memoria, LD y ST.
- Un procesador con E/S mapeada requiere menos lógica interna que uno con E/S independiente, y por tanto suele ser es más barato y rápido (tipo RISC).

EJEMPLO DE MAPEADO DE MEMORIA

CPU Von Neumann

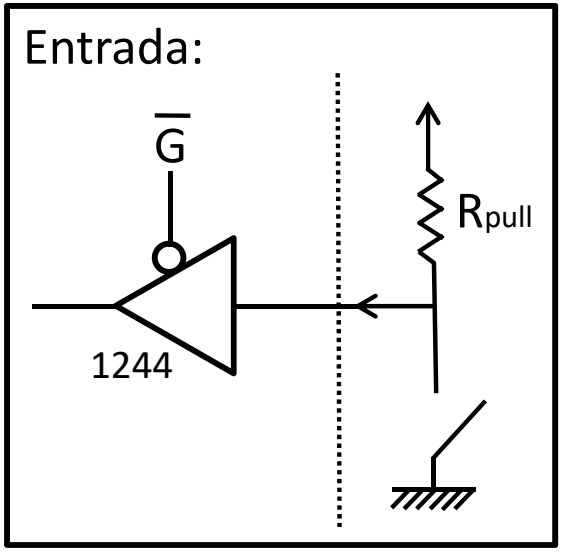
- Bus datos – 8 bits
- Bus direcciones – 16 bits (64 Kbytes)
- Salidas de control de lectura (/RD) y escritura (/WR) activos en baja

Memoria Física (16 Kbytes)

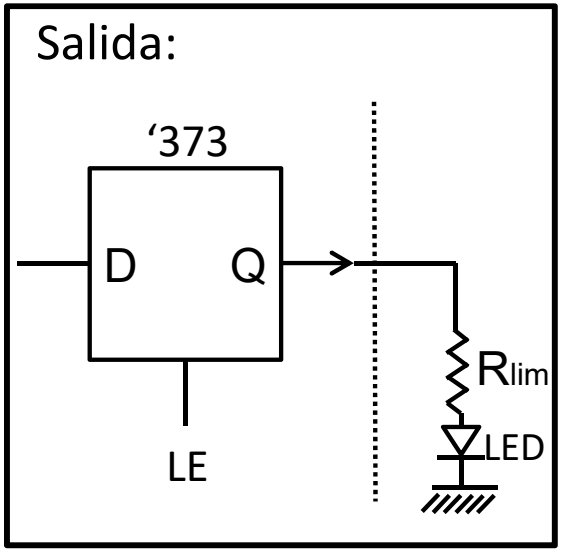
- EPROM – 8 Kbytes (13 bits dirección)
- SRAM – 8 Kbytes (13 bits dirección)
- Chip Select (/CS) activo en baja. Entradas de control de lectura (/OE) y escritura (/WE) activas en baja.

Periféricos

- 1 byte salida
- 1 byte entrada

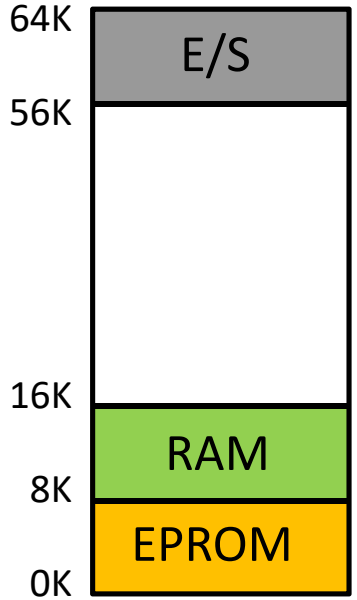


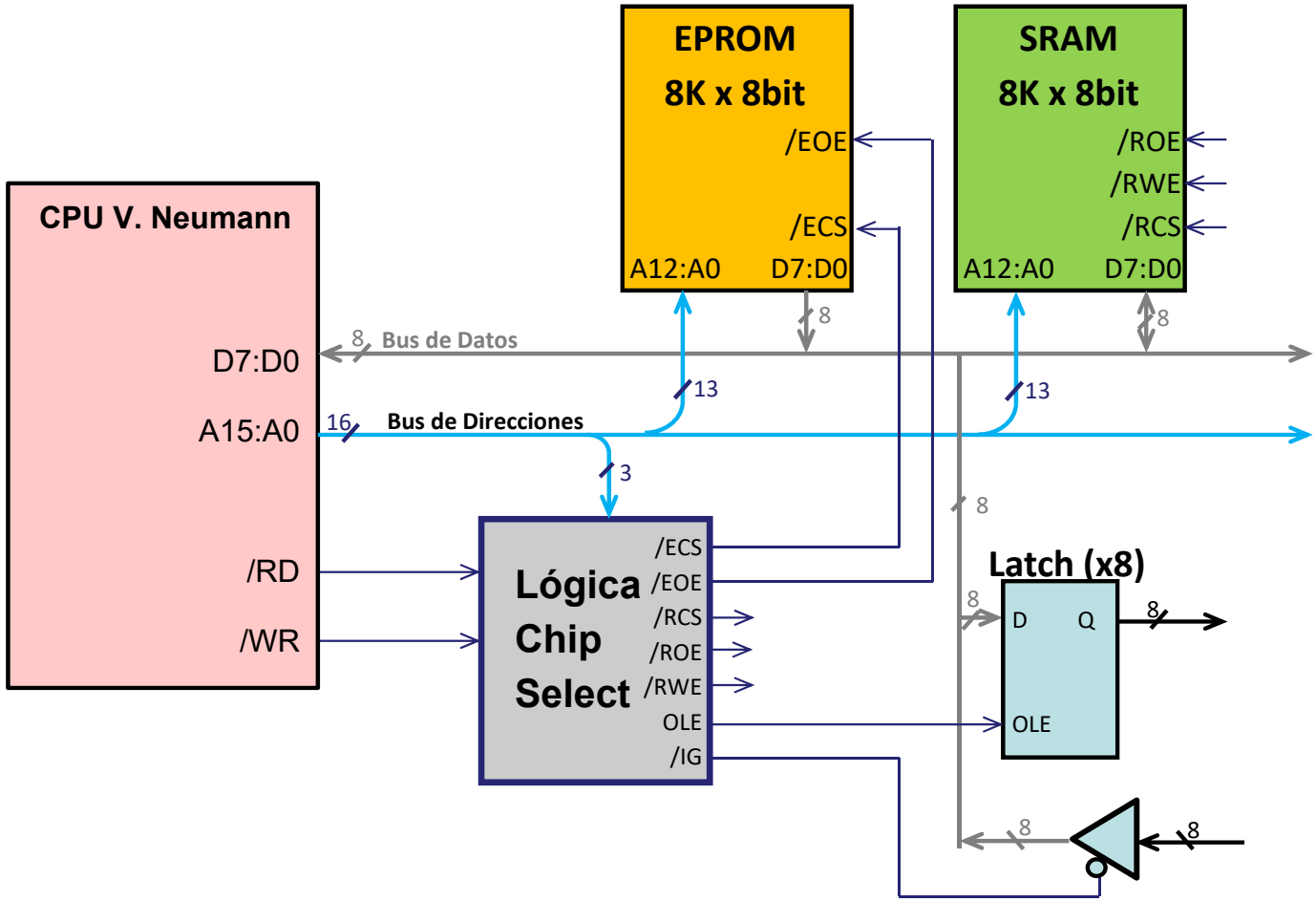
- Entrada de control de lectura (/G) activa en baja



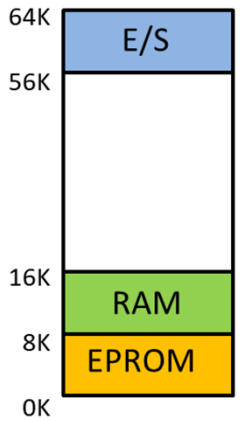
- Entrada de control de escritura (LE) activa en alta

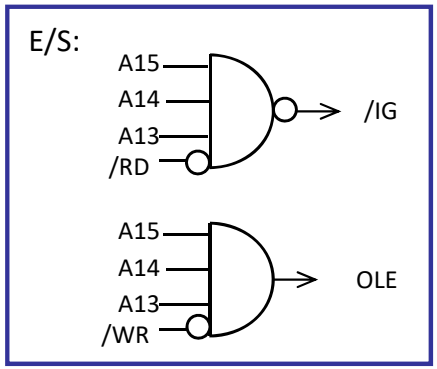
Mapa de Memoria

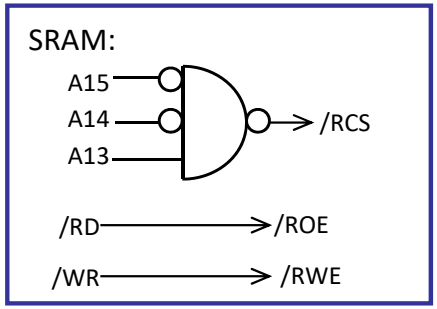


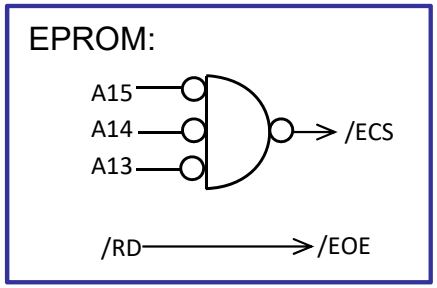


Mapa de Memoria









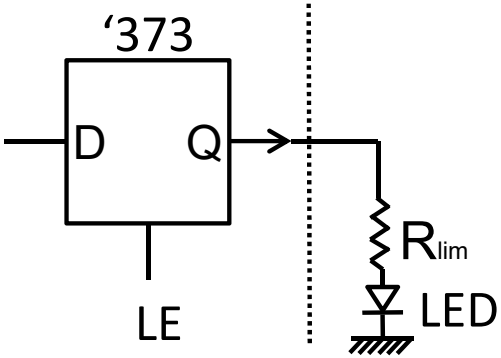
A15 A14 A13	EPROM		SRAM			OUTPUT	INPUT
	/EOE	/ECS	/ROE	/RWE	/RCS	OLE	/IG
000	/RD	0	X	X	1	0	1
001	X	1	/RD	/WR	0	0	1
111	X	1	X	X	1	//WR	/RD
otra	X	1	X	X	1	0	1

Direcciones

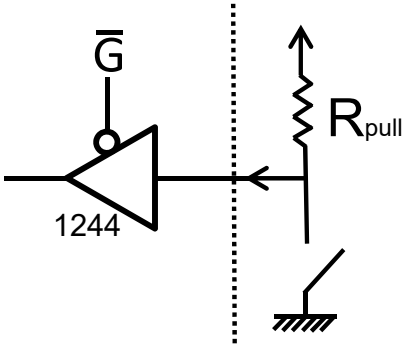
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	-	-	-	-	-								
IO/Mem								Chip Select			Registro				

PERIFÉRICOS SIMPLES

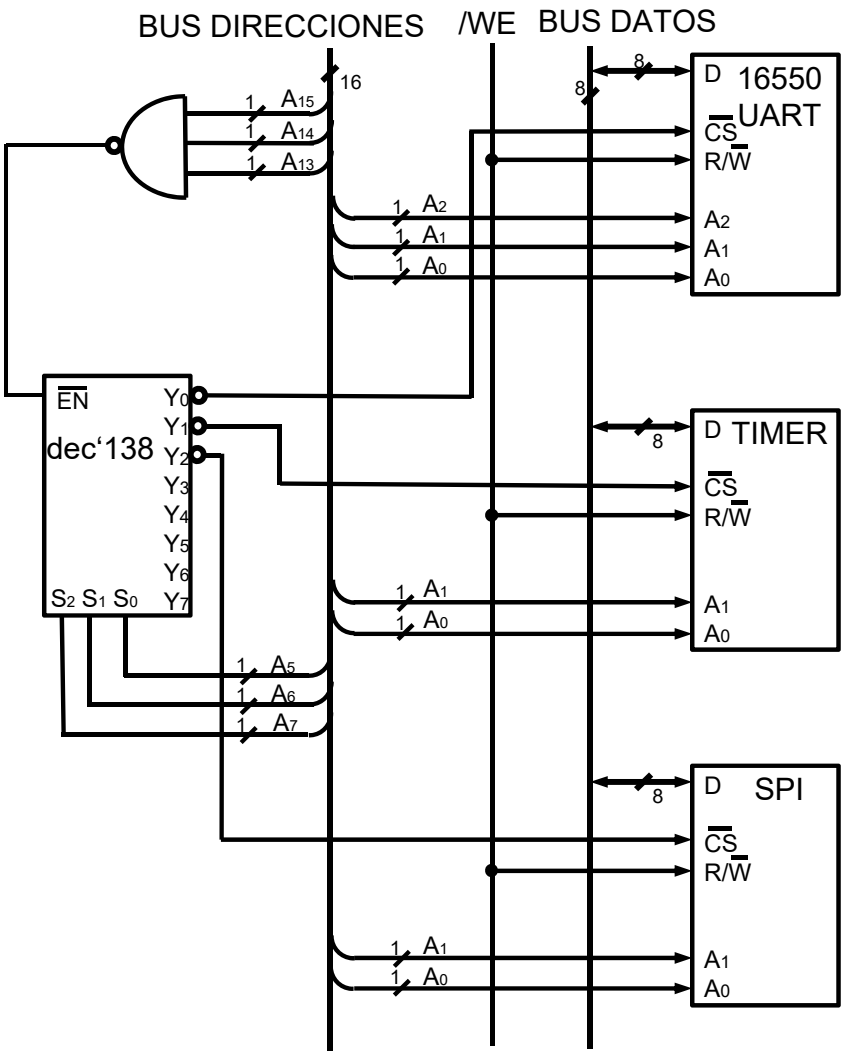
Salida:



Entrada:



DECODIFICACIÓN PERIFÉRICOS COMPLEJOS



TEMA 3: ARQUITECTURA DEL CONJUNTO DE INSTRUCCIONES

3.1.- Conceptos de arquitectura de computación.

3.2.- Direccionamiento de operandos.

3.3.- Modos de direccionamiento.

3.4.- Arquitecturas del conjunto de instrucciones.

3.5.- Instrucciones de transferencia de datos.

3.6.- Instrucciones de manipulación de datos.

3.7.- Instrucciones de control de programa.

3.8.- Interrupciones.

Las instrucciones de manipulación de datos realizan operaciones sobre los datos, proporcionando los recursos de cálculo del procesador. Son de tres tipos:

1º) Instrucciones aritméticas.
2º) Instrucciones lógicas y de manipulación de bits.
3º) Instrucciones de desplazamiento.

INSTRUCCIONES ARITMÉTICAS

Nombre	Nemónico	Descripción
Incremento	INC	Suma uno. Si todos los bits son 1, genera una palabra con todos los bits a 0.
Decremento	DEC	Restar uno. Si todos los bits son 0, genera una palabra con todos los bits a 1.
Suma Resta Multiplicación División	ADD SUB MUL DIV	En procesadores sencillos sólo están disponibles las operaciones de suma y de resta. En esos casos, la multiplicación y la división se tienen que hacer mediante programas. Normalmente estas operaciones están disponibles para diferentes tipos de datos.
Suma con acarreo	ADC	Suma dos operandos teniendo en cuenta el acarreo del cálculo anterior. Permite cálculos de doble precisión.
Resta con acarreo	SBC	Resta dos operandos teniendo en cuenta el acarreo del cálculo anterior. Permite cálculos de doble precisión.
Resta inversa	RSB	Invierte el orden de los operandos, haciendo la resta B-A en lugar de A-B.
Negativo	NEG	Halla el complemento a dos del operando.

INSTRUCCIONES LÓGICAS Y DE MANIPULACIÓN DE BITS

Mediante la aplicación de operaciones lógicas, estas instrucciones permiten la manipulación de bits o de grupos de bits en palabras almacenadas en registros o en memoria.

Normalmente, estas instrucciones tratan cada bit del operando individualmente.

Nombre	Nemónico	Descripción
Poner a 0 (Clear)	CLR	Pone todos los bits del operando a 0.
Poner a 1 (Set)	SET	Pone todos los bits del operando a 1.
Complementar	NOT	Complementa los bits del operando.
Producto lógico Suma lógica OR exclusiva	AND OR EOR	Realizan la función lógica correspondiente entre dos operandos bit a bit. AND se usa con máscaras para poner selectivamente a 0 algunos bits de los operandos, OR para ponerlos a 1, y EOR para complementarlos.
Poner a 0 el acarreo (Clear Carry)	CLRC	Estas instrucciones sobre el bit de acarreo pueden definirse para los demás bits de estado del procesador.
Poner a 1 el acarreo (Set Carry)	SETC	
Complementar el acarreo	COMC	

INSTRUCCIONES DE DESPLAZAMIENTO

Permiten desplazar o rotar los bits de un operando de diversas formas:

Nombre	Nemónico	Descripción
Desplazamiento lógico a la derecha	LSR	
Desplazamiento lógico a la izquierda	LSL	
Desplazamiento aritmético a la derecha	ASR	
Desplazamiento aritmético a la izquierda	ASL	
Rotación a la derecha	ROR	
Rotación a la izquierda	ROL	
Rotación a la derecha extendida	RRX	
Rotación a la izquierda extendida	RLX	

TEMA 3: ARQUITECTURA DEL CONJUNTO DE INSTRUCCIONES

3.1.- Conceptos de arquitectura de computación.

3.2.- Direccionamiento de operandos.

3.3.- Modos de direccionamiento.

3.4.- Arquitecturas del conjunto de instrucciones.

3.5.- Instrucciones de transferencia de datos.

3.6.- Instrucciones de manipulación de datos.

3.7.- Instrucciones de control de programa.

3.8.- Interrupciones.

Las instrucciones de un programa se almacenan en posiciones de memoria consecutivas. Cada vez que se extrae una instrucción de la memoria, el PC se incrementa lo necesario para apuntar a la siguiente instrucción del programa.

Las *instrucciones de control de programa* permiten alterar el flujo de ejecución de los programas actuando sobre el contenido del PC. Con ellas, los procesadores pueden ejecutar diferentes secuencias de instrucciones en función de los resultados de cálculos previos.

Nombre	Nemónico	Descripción
Bifurcación (*Branch*) Salto (*Jump*)	BR JMP	Normalmente se diferencian sólo en el modo de direccionamiento, que suele ser directo o indirecto en los saltos, y relativo en las bifurcaciones. Son instrucciones de una sola dirección, y transfieren la dirección efectiva al PC. Pueden ser incondicionales o condicionales.
Salto implícito (*Skip*)	SKP	Salta la siguiente instrucción de la secuencia si se cumple una determinada condición. Para ello incrementa el PC en su fase de ejecución. No necesita campo de direcciones. Puede combinarse con una instrucción de salto incondicional para hacer una bifurcación condicional.
Comparación	CMP	Realiza la comparación de dos operandos mediante una substracción, pero sin almacenar el valor de la resta. Típicamente, como resultado modifica los bits de estado.
Test mediante AND	TST	Realiza la función AND de dos operandos, pero sin almacenar el resultado. Puede producir los mismos efectos que la instrucción de comparación.
Llamada a subrutina Retorno de subrutina	CALL RET	Instrucciones para el manejo de subrutinas.

EJECUCIÓN CONDICIONAL

Los saltos pueden hacerse condicionados a los bits de estado. Se comprueban sus valores para ver si se cumple una determinada condición: si es cierta se ejecuta el salto, y si no, se pasa a la instrucción siguiente.

Condición	Nemónico	Bits de estado
A < 0, número negativo	MI (*Minus*)	N = 1
A ≥ 0, número positivo o cero	PL (*Positive or zero*)	N = 0
Ha habido desbordamiento	VS (*V set*)	V = 1
No ha habido desbordamiento	VC (*V clear*)	V = 0
A = B, resultado = 0	EQ (*Equal*)	Z = 1
A ≠ B, resultado ≠ 0	NE (*Not equal*)	Z = 0
A ≥ B para números sin signo	CS / HS (*C set* / *Higher or same*)	C = 1
A < B para números sin signo	CC / LO (*C clear* / *Lower*)	C = 0
A > B para números sin signo	HI (*Higher*)	C = 1 y Z = 0
A ≤ B para números sin signo	LS (*Lower or same*)	C = 0 o Z = 1
A > B para números con signo	GT (*Greater than*)	Z = 0 y N = V
A ≥ B para números con signo	GE (*Greater or equal*)	N = V
A < B para números con signo	LT (*Less than*)	N ≠ V
A ≤ B para números con signo	LE (*Less than or equal*)	Z = 1 o N ≠ V

En algunos procesadores todas las instrucciones se pueden hacer de ejecución condicional.

SUBRUTINAS

Es relativamente frecuente que conjuntos de instrucciones en un programa se repitan. En lugar de copiar varias veces en memoria las mismas secuencias de instrucciones, se almacenan una única vez en lo que se denomina *subrutina*. Cada vez que se necesite ejecutar esa secuencia se hace un salto a la primera instrucción de la subrutina. Cuando se termine de ejecutar la subrutina, se hace otro salto para volver al programa principal.

La instrucción de llamada a subrutina CALL posee un campo de dirección y realiza dos operaciones:

1ª) Guarda el valor del PC en una zona de almacenamiento temporal (dirección de retorno).

2ª) La dirección de la instrucción CALL se carga en el PC (primera instrucción de la subrutina).

La última instrucción de una subrutina es la de retorno al programa principal RET: carga en el PC la dirección almacenada por la instrucción CALL.

Algunos procesadores utilizan una pila como almacenamiento temporal de las direcciones de retorno, lo cual permite manejar *subrutinas anidadas*:

CALL dir:	SP ← SP-1	Decrementa el puntero de pila.
	M[SP] ← PC	Almacena la dirección de retorno en la pila.
	PC ← dir	Transfiere el control a la subrutina.
RET:	PC ← M[SP]	Transfiere la dirección de retorno al PC.
	SP ← SP+1	Incrementa el puntero de pila.

TEMA 3: ARQUITECTURA DEL CONJUNTO DE INSTRUCCIONES

3.1.- Conceptos de arquitectura de computación.

3.2.- Direccionamiento de operandos.

3.3.- Modos de direccionamiento.

3.4.- Arquitecturas del conjunto de instrucciones.

3.5.- Instrucciones de transferencia de datos.

3.6.- Instrucciones de manipulación de datos.

3.7.- Instrucciones de control de programa.

3.8.- Interrupciones.

DEFINICIÓN DE INTERRUPCIÓN

Una *interrupción* transfiere el control del programa que está en ejecución a otro programa de servicio como resultado de una petición, ya sea interna o externa al procesador. El control debe volver al programa original una vez se ha ejecutado el programa de servicio.

El tratamiento de las interrupciones es similar al que se hace con las subrutinas, salvo que:

1º) Las interrupciones se pueden producir de forma imprevisible.

2º) La dirección del programa de servicio se suele determinar por hardware.

3º) Se almacena no sólo el valor del PC, sino también los registros del procesador.

Generalmente, el tratamiento de las interrupciones se hace dentro de un modo de funcionamiento específico del procesador que es distinto del de usuario o supervisor.

También se utilizan interrupciones para solicitar cambios en el modo de funcionamiento del procesador. Por ejemplo, cuando desde un programa de usuario se solicita permiso al sistema operativo para poder ejecutar algún tipo de instrucción privilegiada sólo accesible en el modo supervisor.

El PSR, además de almacenar el estado del procesador y de los banderines, suele incluir información sobre qué tipo de interrupciones están permitidas o habilitadas.

TIPOS DE INTERRUPCIONES

Los principales tipos de interrupciones que provocan una ruptura en la ejecución normal de un programa son:

1º) Interrupciones externas.

Pueden proceder de:

- Dispositivos de E/S, que solicitan o acaban una transferencia de datos.
- Dispositivos de temporización, cuando un determinado evento no finaliza en el tiempo esperado (*time-out*).
- La detección de un fallo de alimentación. El servicio puede incluir instrucciones para almacenar la información relevante del procesador en un medio no volátil (disco magnético).

2º) Interrupciones internas.

Reciben también el nombre de *excepciones*. Pueden deberse a:

- Desbordamiento aritmético. Ej.: división por cero.
- Desbordamiento de memoria. Ej.: se excede el tamaño máximo de la pila, violación de una protección, puntero desbocado.
- Código de operación no válido.

3º) Interrupciones software.

Mientras que los dos tipos anteriores son iniciados por el hardware del procesador, una interrupción software se inicia mediante la ejecución de una instrucción. Se emplean, por ejemplo, para cambiar del modo usuario al modo supervisor y así solicitar algún servicio gestionado por el sistema operativo.

TRATAMIENTO DE LAS INTERRUPCIONES

1º) Se termina de ejecutar la instrucción del programa en curso.

2º) Antes de acceder a la siguiente instrucción del programa, se comprueba si ha habido alguna petición de interrupción.

3º) Si se ha producido, se pasa a un ciclo de interrupción hardware:

   a) Se deshabilita el servicio de interrupciones (si no se quiere que la atención a la interrupción se pueda ver a su vez interrumpida por otra nueva petición de servicio).

   b) Se almacena el valor del PC y de los registros del procesador en la pila.

   c) Se transfiere al PC la dirección de comienzo de la rutina que da servicio a la interrupción (*vector de interrupción*).

4º) La última instrucción de la rutina de servicio es de retorno:

   a) Se extrae de la pila la dirección de retorno al programa en curso y se transfiere al PC, así como el resto de información de los registros del procesador.

   b) Se habilita nuevamente el servicio de interrupciones.